Week 5 Practice Problems

Problem 1 : Walking through Quicksort's in-place partition

Recall the in-place partitioning process from lecture. Work through this process on $A = [6, 3, 8, 2, 9, 5, 7, 4]$ (pivot $= A[8] = 4$). For each value of j from 1 to 7, show whether $A[j] \leq x$, what happens to i, and the resulting array. Then show the final swap that places the pivot, and confirm the array is correctly partitioned around 4.

Solution. Pivot $x = A[8] = 4$. Start: $i = 0$, $A = [6, 3, 8, 2, 9, 5, 7, 4]$.

j	$A[j]$	$\leq x$?	Result
1	6	no	leave in place: $[6, 3, 8, 2, 9, 5, 7, 4]$
2	3	yes	$i \to 1$, swap $A[1], A[2]$: $[3, 6, 8, 2, 9, 5, 7, 4]$
3	8	no	leave in place: $[3, 6, 8, 2, 9, 5, 7, 4]$
4	2	yes	$i \to 2$, swap $A[2], A[4]$: $[3, 2, 8, 6, 9, 5, 7, 4]$
5	9	no	leave in place: $[3, 2, 8, 6, 9, 5, 7, 4]$
6	5	no	leave in place: $[3, 2, 8, 6, 9, 5, 7, 4]$
7	7	no	leave in place: $[3, 2, 8, 6, 9, 5, 7, 4]$

Final step: swap $A[i{+}1] = A[3]$ with the pivot $A[8]$: $[3, 2, \mathbf{4}, 6, 9, 5, 7, 8]$.

Final check: the low part of the array $A[1{:}2] = [3, 2]$ are all ≤ 4, and the high part of the array $A[4{:}8] = [6, 9, 5, 7, 8]$ are all > 4, with the pivot 4 correctly placed at index 3 in between the two parts of the array. The array is correctly partitioned.

Problem 2 : Indicator random variables and expectation

Let π be a permutation of $\{1, 2, \ldots, n\}$ chosen uniformly at random (where $\pi(i)$ is the value at position i). Say position i is a **fixed point** if $\pi(i) = i$. Let X be the total number of fixed points in π.

(a) For $i = 1, \ldots, n$, let X_i be the indicator random variable for the event that position i is a fixed point. What is $P(X_i = 1)$?

(b) Express X in terms of $X_1, \ldots, X_n$, and use linearity of expectation to find $E[X]$.

(c) The X_i's are not independent (e.g., knowing $\pi(1) = 1$ changes the probability that $\pi(2) = 2$, since now only $n - 1$ values remain for the other positions). Does that cause a problem for the computation in part (b)? Why or why not?

Solution. (a) $\pi(i)$ is equally likely to be any of the n values $\{1, \ldots, n\}$, so $P(\pi(i) = i) = 1/n$. Thus, $P(X_i = 1) = 1/n$.

(b) $X = X_1 + X_2 + \cdots + X_n$ (total count of fixed points is sum of the indicators). By linearity of expecation:

$$E[X] = \sum_{i=1}^{n} E[X_i] = \sum_{i=1}^{n} \frac{1}{n} = n \cdot \frac{1}{n} = 1.$$

This means that no matter how large n is, the expected number of fixed points in a random permutation is exactly 1.

(c) Linearity of expectation, $E[X_1 + \cdots + X_n] = E[X_1] + \cdots + E[X_n]$, holds regardless of whether the X_i's are independent. The dependence between the X_i's would matter if we were trying to compute something like $\mathrm{Var}(X)$ or $P(X_1 = 1 \text{ and } X_2 = 1)$ directly, but it has no effect on this computation of $E[X]$.

Problem 3 : Unbalanced Quicksort claim

Lecture states that if Quicksort's pivot always splits the subarray into a $1/10 : 9/10$ ratio, the resulting recurrence

$$T(n) = T(n/10) + T(9n/10) + \Theta(n)$$

is *still* $\Theta(n \log n)$, the same as an exactly-balanced $1/2 : 1/2$ split. However, lecture only asserts this without actually doing so. Let's verify it.

(a) In the recursion tree, consider the path that always follows the smaller $(n/10)$ branch. How many levels does it take for the subproblem size along this path to shrink to (roughly) 1? Express your answer in Θ notation.

(b) Now consider the path that always follows the larger $(9n/10)$ branch. How many levels does it take for this path to shrink to (roughly) 1? Again express your answer in Θ notation.

(c) Explain why both of your answers to (a) and (b) are $\Theta(\log n)$ despite having different constants. Also explain why this means the recursion tree has $\Theta(\log n)$ levels totals.

(d) Each level of the tree has total cost $O(n)$. Combine this with part (c) to conclude $T(n) = O(n \log n)$. Then, argue that there is a matching lower bound $T(n) = \Omega(n \log n)$.

Solution. (a) The size after d levels is $n \cdot (1/10)^d$. Setting this ≈ 1 gives $d = \log_{10} n = \Theta(\log n)$.

(b) The size after d levels is $n \cdot (9/10)^d$. Setting this ≈ 1 gives $d = \log_{10/9} n = \dfrac{\ln n}{\ln(10/9)} = \Theta(\log n)$.

(c) Both expressions are of the form $\frac{\ln n}{\ln b}$ for some fixed constant $b > 1$ ($b = 10$ or $b = 10/9$), and $\frac{1}{\ln b}$ is just a constant. This means that both are $\Theta(\log n)$, just with different constant factors ($\log_{10/9} n$ is numerically much larger than $\log_{10} n$, since $10/9$ is much closer to 1 than 10 is, meaning the slow branch takes many more levels to shrink away). We note that the tree can also stop growing once every single branch has reached its base case, and the slow $(9n/10)$ branches will take the longest to do so. Thus, the overall height of the tree is controlled by part (b)'s length which is indeed still $\Theta(\log n)$.

(d) Upper bound: each of the $\Theta(\log n)$ levels contributes $O(n)$ total work (summed across

all nodes existing at that level), so the total is $\Theta(\log n) \cdot O(n) = O(n \log n)$.

Lower bound: during the first $d = \log_{10} n = \Theta(\log n)$ levels (part (a)'s length), no branch has reached its base case yet (even the fastest shrinking branch is still ≥ 1 in size until that point), so every one of those levels has total size exactly n, and hence contributes $\Theta(n)$ work. That alone gives at least $\Theta(\log n) \cdot \Theta(n) = \Theta(n \log n)$ work.

Combining both bounds gives us $T(n) = \Theta(n \log n)$.

Problem 4 : Generalizing the box analogy

Recall in lecture, you discussed a nice example for where randomization helps.

The example is as follows: with 2 boxes, one containing \$100 and the other \$0, a deterministic "always pick Box 1" strategy can result in getting \$0 in the worst case, while a randomized coin flip strategy guarantees an expected \$50 no matter where the money is hidden.

Now generalize to k boxes: \$100 is placed in exactly one of the k boxes (and \$0 in the rest) before you choose any boxes, and you don't know which one holds the money.

(a) If you use the deterministic strategy "always choose Box 1," what is the worst case amount you receive, regardless of where the money is placed?

(b) If you use the randomized strategy and choose uniformly at random among the k boxes, what is your expected gain? Explain why this value is the same regardless of which specific box the money has been placed in.

(c) As k grows large, what happens to the deterministic strategy's worst case result? What happens to the randomized strategy's guaranteed expected gain?

(d) Explain the connection to Randomized Quicksort. In what sense is choosing a pivot position analogous to choosing a box here, and what does this example suggest about why Randomized Quicksort avoids the guaranteed $\Theta(n^2)$ worst case that the deterministic (always choose the last element as the pivot) version experiences on a sorted array?

Solution. (a) If the money is placed in any box other than Box 1, then the worst case gain is \$0.

(b) Each box is chosen with probability $1/k$, so regardless of which box actually holds the \$100, the expected gain is $\frac{1}{k} \cdot 100 + \frac{k-1}{k} \cdot 0 = \frac{100}{k}$. This doesn't depend on which box the money is in, because the money is placed before you choose a box. From the perspective of the person hiding the money, every box is equally likely to be the one you pick, so every placement gives the same expected gain.

(c) The deterministic strategy's worst case stays exactly \$0 forever regardless of the value of k. The randomized strategy's expected gain, $100/k$, moves toward 0 as $k \to \infty$, but it is always strictly positive. However, this expected gain never collapses to the deterministic strategy's \$0 worst case.

(d) Choosing which element of the array to use as the pivot is analogous to choosing which box to open. A deterministic rule like always picking the last element in the array can be easily defeated by a specific input (e.g. a sorted array). Similarly, picking a deterministic rule like "always pick Box 1" can be easily defeated by placing the money in any box other than Box 1. Choosing the pivot uniformly at random instead is like randomizing which box you pick. No matter what the input array looks like, the expected outcome (here, expected running

time) is controlled by an average over the randomized choice, rather than the worst possible randomized outcome. This means that there is no single fixed input that can force bad expected performance, exactly as no fixed box placement can force the randomized expected gain down to \$0.

Problem 5 : How tight is the $\Omega(n \log n)$ lower bound, concretely?

In lecture, you proved that any comparison sort algorithm requires $\Omega(n \log n)$ comparisons in the worst case, using the fact that its decision tree must have at least $n!$ leaves, and a binary tree with $n!$ leaves has height at least $\log_2(n!)$. Let's look more closely at this for a small value of n.

(a) For $n = 4$, compute $4!$ and $\log_2(4!)$.

(b) A decision tree of height h has at most 2^h leaves. Using your answer to (a), find the smallest integer h such that $2^h \geq 4!$. What does this tell you about the minimum possible worst case number of comparisons needed to sort 4 elements?

(c) Insertion Sort's worst case occurs on a reverse sorted input where the i-th element (for $i = 2, \ldots, n$) must be compared against all $i-1$ elements already placed before it stops (or reaches the front). Using this, compute the exact worst case number of comparisons Insertion Sort performs on 4 elements. Is it equal to your lower bound from (b)?

(d) Given your answers to (b) and (c), do you think the $\Omega(n \log n)$ bound is always possible with some comparison sort algorithm, for every n? What would it mean about an algorithm if its worst case matched the bound exactly?

Solution. **(a)** $4! = 24$, and $\log_2(24) \approx 4.585$.

(b) We need the smallest integer h with $2^h \geq 24$. Since $2^4 = 16 < 24$ and $2^5 = 32 \geq 24$, we get $h = 5$. So any comparison sort algorithm must perform at least 5 comparisons in the worst case to sort 4 elements.

(c) On a reverse sorted array of 4 elements, element 2 is compared against 1 earlier element, element 3 against 2 earlier elements, and element 4 against 3 earlier elements (each comparison fails until the new element reaches the front, so all are performed). This leaves us with $1+2+3 = 6$ comparisons, which is strictly greater than the lower bound of 5 from part (b). Thus, Insertion Sort is not optimal in the worst case for $n = 4$.

(d) The bound from part (b) is a lower bound that holds for every comparison sort algorithm. Insertion Sort is one example of an algorithm that doesn't meet it exactly. However, the bound can be tight. For $n = 4$, it turns out Merge Sort's worst case is exactly 5 comparisons, which matches the lower bound precisely. An algorithm whose worst case matches $\log_2(n!)$ (rounded up) exactly, for a given n, is using every comparison as efficiently as an algorithm possibly can. No comparison sort algorithm could do better on n, in the worst case.